

Lab assignment 2: Transitive clauses

Introduction to programming (LIS020)

Robert Östling
robert@ling.su.se

Amanda Kann
amanda.kann@ling.su.se

July 10, 2026

Purpose and learning outcomes

In this assignment, you will use an external library (`pyconll`) to read data from the Universal Dependency Treebanks and extract information using its syntactic annotations. You will learn how to use external libraries, where *reading documentation* is a crucial skill. You will also practice using virtual environments.

To achieve a given grade for this assignment, you will need to:

- **Pass:** Read relevant documentation for the `pyconll` library and implement code to search for sequential patterns in the language data.
- **Pass with distinction:** In addition to the above, also independently read documentation for the part of the `pyconll` library dealing with tree structures, and implement more advanced code to search for tree patterns in the language data. You should also follow the standards for code documentation outlined in the end of this document.

Submitting your solution

Submit the file `extract.py` to Canvas.

If you are working on the `triton` server, you can copy files from there to your local computer by executing the following command on **your** computer (not while logged into `triton`):

```
scp abcd1234@triton.dsv.su.se:lis020/lab2/extract.py ./
```

Background

The Universal Dependency Treebanks consist of syntactically and morphologically annotated data from 179 (as of September 2025) different languages. The annotations have been harmonized across languages to make

conceptually similar structures receive similar annotations across languages, and the same CoNLL-U file format is used across all the data, which facilitates multilingual tools and research.

You do not need to learn all the details about Universal Dependencies or the CoNLL-U file format, but we do encourage you to read up on the basics as these standards have become very widespread in modern Natural Language Processing:

- <https://universaldependencies.org/>
- <https://universaldependencies.org/format.html>

Tasks

The overall goal in this lab assignment is to write code to extract transitive clauses from syntactically annotated data. You will use the `pyconll` library to read CoNLL-U files from the Universal Dependencies Treebanks, and at a minimum use the following (very) simplistic assumption: any sequence of noun-verb-noun (as indicated by the universal part of speech tags) is a transitive clause, since you will be working with English which mainly uses subject-verb-object order and can have bare nouns as verb arguments.

To pass with distinction, you will additionally need to write code to search through the tree structure, so that clauses with other word orders or with additional words (like articles or adjectives) inserted between the verb and the nouns can also be found. Note that we still use a simplistic assumption here (the subject and object of a transitive clause may be something other than a noun), but we relax the assumption of subject-verb-object word order. In addition to using parts of speech to identify the subject and object, you will use the dependency labels present in the Universal Dependencies Treebanks.

Set up a virtual environment

You will need to install the `pyconll` library yourself, and for this you will need to create a *virtual environment* for Python. Each virtual environment allows you to customize Python and install a particular selection of external libraries, without interfering with other virtual environments. In case you are curious about why this is needed, search the web for “dependency hell”.

To create a virtual environment called “lab2”, you can do the following:

```
mkdir -p ~/venv
python3 -m venv ~/venv/lab2
```

The above only has to be performed once. The files required by the virtual environment will now be located in the subdirectory `venv/lab2` inside

your home directory. You should never need to directly go to this directory or do anything with the files there.

In order to *use* the virtual environment, you need to activate it:

```
source ~/venv/lab2/bin/activate
```

After this, your prompt should look like this to indicate that the “lab2” virtual environment is now active:

```
(lab2) robe@triton:~$
```

Now, everything you install with the Python installer, `pip`, will be installed in this virtual environment. Also, if you launch Python from this virtual environment, you can import these libraries. Start with installing the `pyconll` library:

```
pip install pyconll==3.3.1
```

(**Note:** version 4 of `pyconll` introduces breaking changes and only works on very recent Python versions. The specifier `==3.3.1` makes sure that you install a `pyconll` version that is compatible with this assignment.)

A very common mistake is to forget loading the virtual environment before running the program with Python. This will cause the program to crash with an error message like this:

```
Traceback (most recent call last):
  File "extract.py",
    line 2, in <module>
      import pyconll
ModuleNotFoundError: No module named 'pyconll'
```

Sequential patterns (mandatory)

Start by copying the code template to your own home directory:

```
mkdir -p ~/lis020/lab2
cd ~/lis020/lab2
cp /home/dsv/robe/lis020/labs/transitive/extract.py ./
```

This program contains a `main` function which performs some tests. In order to pass this lab, at a minimum you need to add the missing code in the function `find_transitive_linear`. Carefully read the docstring so that you understand what the function is expected to take as arguments and return.

You will need to read the `pyconll` documentation on checking the elements of sentences, which are represented by the `Sentence` class there:

- <https://pyconll.readthedocs.io/en/stable/pyconll/unit/sentence.html>

Note that the (universal) part of speech tag that you need to check is stored in the `upos` attribute of the `Token` objects contained inside a `Sentence` object. The `Token` class is documented here:

- <https://pyconll.readthedocs.io/en/stable/pyconll/unit/token.html>

Note: as the documentation in the code says, your code should *only* look at part of speech tags, not the dependency labels. That comes in the Pass with distinction task below, if you choose to attempt it.

Tree-structured patterns (pass with distinction)

Add the missing code in the functions `tree_search` and `extract_transitive`. Note that the relatively advanced features of function arguments and recursive function definitions are used here, so you may want to get back to the literature or lecture notes to read up on that before continuing. In addition, this requires you to independently read the `pyconll` documentation on working with tree-structured data.

Unlike the sequential patterns, you also need to take the dependency relation types into account. Universal Dependencies uses the label "`nsubj`" for subjects, and "`obj`" for objects. This information is stored in the `deprel` attribute of `Token` objects representing the *dependent* words, i.e. the head noun of the subject or object.

Documentation requirements

- Docstrings for all functions, following the Google conventions (this is what is used in the pre-filled part of the code template, so unless you add additional functions this has already been taken care of).
- In-line comments when appropriate.
- Appropriate identifier names.
- We do **not** demand type annotations, for the reason that type annotations with callable functions and classes from external libraries gets fairly complex and messy. You are of course welcome to add them if you want.